Fondamenti di Grafica Tridimensionale

Laurea Magistrale in Ingegneria Informatica e delle Tecnologie dell'Informazione

Docente Dr. Ugo Erra

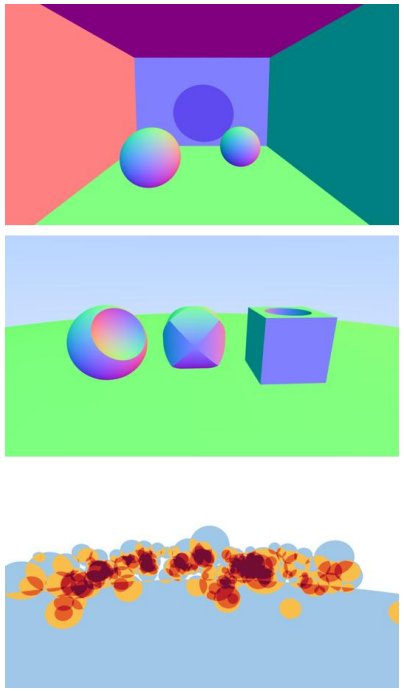
I° Homeworks

Homeworks

- Nuove primitive
- La camera
- Qualità e prestazioni
- Modalità di consegna

Come funziona

- Si parte dal progetto della **lezione 5**: camera con `lookfrom`, `lookat`, `vup`, `vfov`, antialiasing e `parallel_render`
- Ogni scheda dice **che cosa fare**, la matematica essenziale e **che cosa consegnare**
- L'indicatore in alto a destra dà la difficoltà: **facile** (si estende il codice che avete), **medio** (una formula nuova e i suoi casi limite), **difficile** (più pezzi nuovi da far funzionare insieme)
- Le forme nuove si scrivono come sottoclassi di `hittable` e usano `hit_record`, `interval` e `set_face_normal` della lezione 4
- Niente librerie esterne per i test di intersezione: il test lo scrivete voi
- Finché il test non è corretto lavorate a 200×112 pixel e un solo campione: si aspetta molto meno



tre risultati possibili: una stanza di quadrilateri, tre solidi CSG, una mappa di calore

Il colore, per ora, viene solo dalle normali

- materiali e luci arrivano dalla lezione 6: fino ad allora un oggetto si colora con `0.5 * (n + 1)`, come nella lezione 3
- è un vantaggio: se la normale è sbagliata, l'immagine ve lo dice subito

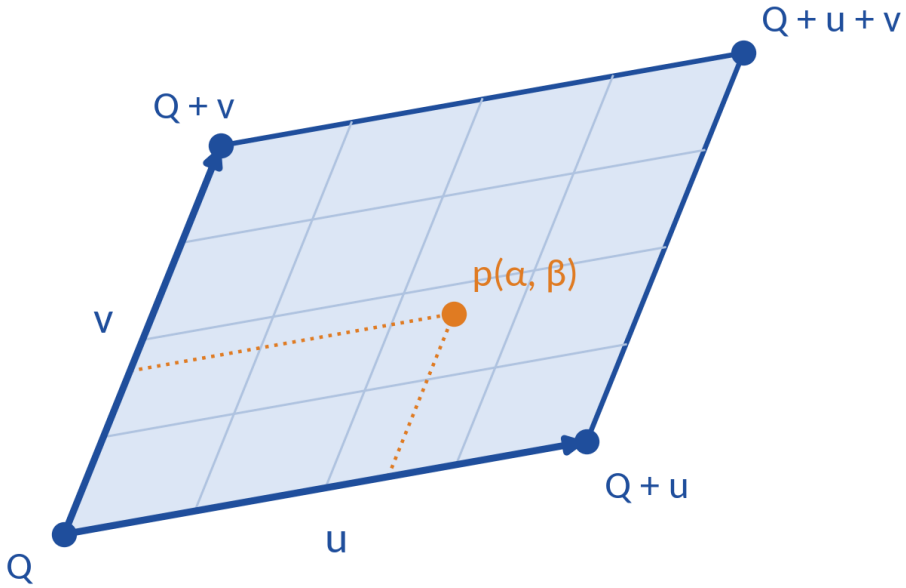
Homeworks

- **Nuove primitive**
- La camera
- Qualità e prestazioni
- Modalità di consegna

1 · Come si definisce un quadrilatero

●○○ facile

- Servono **tre** cose: un vertice **Q** e i due vettori lato **u** e **v** che partono da lì
- Gli altri tre vertici non si danno: si ricavano come **Q + u**, **Q + v** e **Q + u + v**
- Ogni punto si scrive come **Q + α u + β v**, con α e β fra 0 e 1: sono le sue due coordinate sul quadrilatero
- Cambiando solo l'intervallo ammesso per (α, β) si ottengono anche il **triangolo** e il **disco**
- **u** e **v** non devono essere paralleli, altrimenti il quadrilatero è degenere



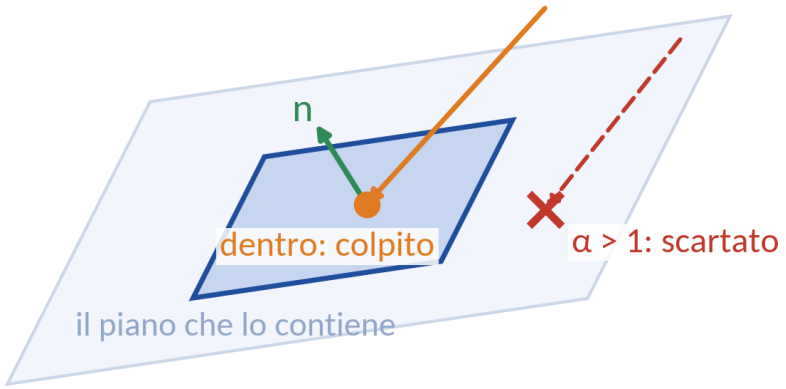
α e β variano fra 0 e 1

$$\mathbf{p}(\alpha,\beta) = \mathbf{Q} + \alpha\,\mathbf{u} + \beta\,\mathbf{v}, \qquad \alpha,\beta \in [0,1]$$

tre parametri: un vertice e due vettori lato

1 · Come si calcola l'intersezione

- Il quadrilatero sta su un piano: prima si interseca **quel piano**, poi si guarda se il punto ci sta dentro
- La normale è **n = u × v**, e il piano è l'insieme dei punti con **n · p = D**
- Sostituendo il raggio si ricava **t** con una sola divisione. Se **n · d** è quasi zero il raggio è parallelo: si scarta **prima** di dividere
- Trovato il punto, le sue (α, β) si ottengono con due prodotti vettoriali, senza risolvere un sistema



prima il piano, poi le due coordinate (α, β)

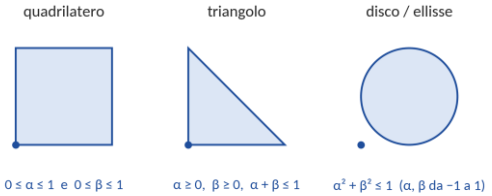
$$\mathbf{n} = \mathbf{u} \times \mathbf{v}, \qquad D = \mathbf{n} \cdot \mathbf{Q}, \qquad t = \frac{D - \mathbf{n} \cdot \mathbf{o}}{\mathbf{n} \cdot \mathbf{d}}$$

$$\alpha = \mathbf{w} \cdot \big((\mathbf{p} - \mathbf{Q}) \times \mathbf{v}\big), \qquad \beta = \mathbf{w} \cdot \big(\mathbf{u} \times (\mathbf{p} - \mathbf{Q})\big), \qquad \mathbf{w} = \frac{\mathbf{n}}{\mathbf{n} \cdot \mathbf{n}}$$

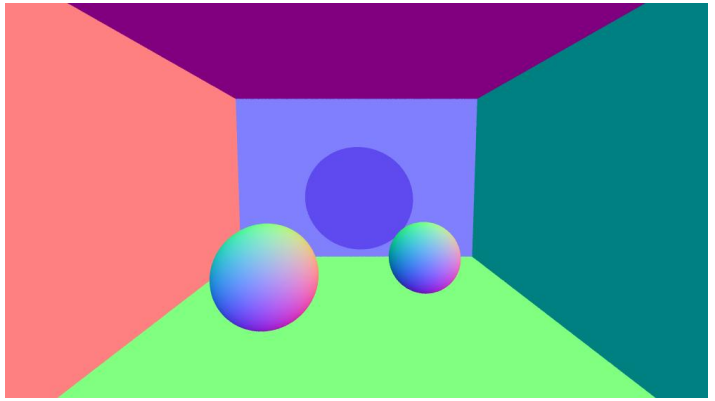
il vettore **w = n / (n · n)** serve solo a normalizzare i due prodotti

1 · Una stanza fatta di quadrilateri

- Scrivere `quad : hittable` con `Q`, `u`, `v` e il test su (α, β)
- Ricavarne `disk` e `triangle` cambiando **solo** la condizione di appartenenza
- Comporre una stanza chiusa: pavimento, soffitto e tre pareti, più un disco inclinato appeso al fondo
- Mettere la camera **dentro** la stanza: è anche un buon test della convenzione sulle normali della lezione 4



tre forme, un solo test



cinque quadrilateri, un disco e due sfere

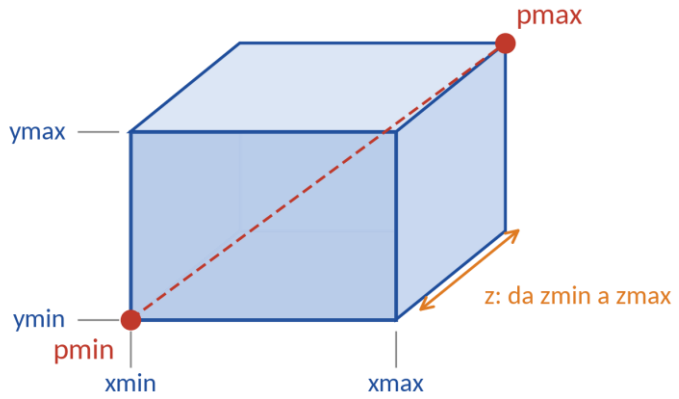
Che cosa consegnare

- `quad`, `disk` e `triangle` come sottoclassi di `hittable`
- due render: la stanza vista da dentro e una forma a scelta vista **di taglio**, per mostrare che il caso parallelo è gestito

2 · Come si definisce una scatola

●●○ medio

- Bastano **due punti opposti**: `pmin` e `pmax`
- È l'insieme dei punti compresi fra i due, **componente per componente**: x fra xmin e xmax, e così per y e z
- Le sei facce non si memorizzano: sono i piani `x = xmin`, `x = xmax` e gli altri quattro
- Vista in un altro modo è l'**intersezione di tre lastre** (slab), una per asse
- «Allineata agli assi» vuol dire che i lati sono paralleli a x, y e z: è questo che rende il test così veloce



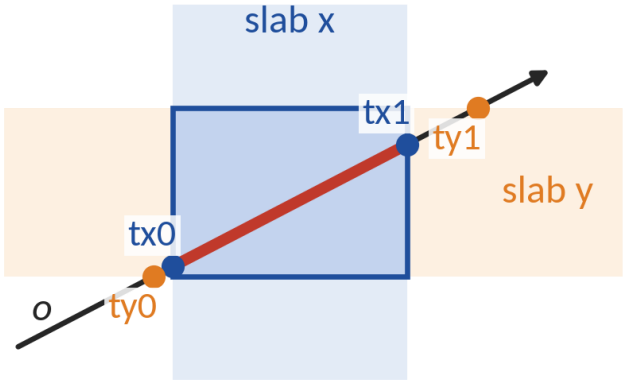
sei facce, ognuna su un piano x, y o z costante

$$\text{il box} = \{\mathbf{p} \ : \ \mathbf{p}_{\min} \le \mathbf{p} \le \mathbf{p}_{\max} \text{ componente per componente}\}$$
$$= \underbrace{[x_{\min}, x_{\max}]}_{\text{slab } x} \times \underbrace{[y_{\min}, y_{\max}]}_{\text{slab } y} \times \underbrace{[z_{\min}, z_{\max}]}_{\text{slab } z}$$

due punti, sei facce, nessuna rotazione

2 · Il metodo delle slab

- Per ogni asse il raggio entra nella lastra a `t0` e ne esce a `t1`: due divisioni
- Il raggio colpisce il box se i tre intervalli si **sovrappongono** tutti insieme
- Basta tenere il massimo degli ingressi e il minimo delle uscite
- La divisione per `d = 0` non va evitata con un `if`: con gli infiniti IEEE il conto torna da solo, ed è più veloce
- La normale è quella della faccia che ha prodotto `t_in`: l'asse dice quale, il segno di `d` dice il verso



[t_in , t_out] = intersezione degli intervalli dei tre assi

$$t_a = \frac{\mathrm{lo}_a - \mathbf{o}_a}{\mathbf{d}_a}, \qquad t'_a = \frac{\mathrm{hi}_a - \mathbf{o}_a}{\mathbf{d}_a}, \qquad a \in \{x, y, z\}$$

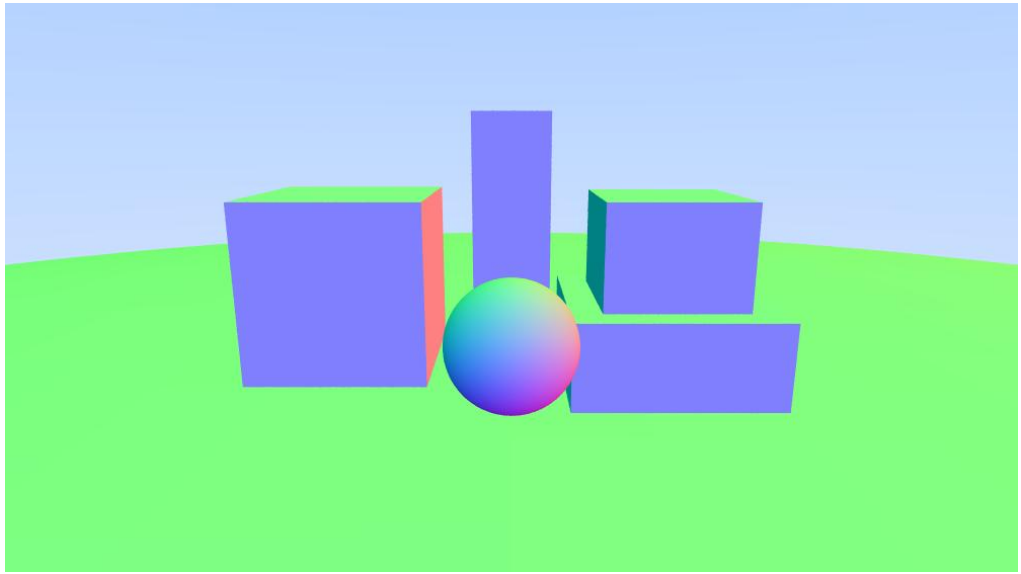
$$t_{\mathrm{in}} = \max_a \min(t_a, t'_a), \qquad t_{\mathrm{out}} = \min_a \max(t_a, t'_a)$$

$$\text{colpito se } \; t_{\mathrm{out}} > t_{\mathrm{in}} \;\text{ e }\; t_{\mathrm{out}} > 0$$

Williams, Barrus, Morley, Shirley (2005), **An Efficient and Robust Ray-Box Intersection Algorithm**

2 · La primitiva più importante del corso

- Scrivere `box : hittable` con il metodo delle slab e la normale corretta su tutte e sei le facce
- Usare la classe `interval` della lezione 4 per tenere [`t_in`, `t_out`]: è esattamente il suo mestiere
- Comporre una scena con almeno tre box e una sfera, inquadrati da un angolo che mostri tre facce diverse
- **Da qui si passa alla lezione 9**: il bounding volume di un BVH è proprio un box, e questo test viene eseguito milioni di volte al secondo



tre box e una sfera, colore dalle normali

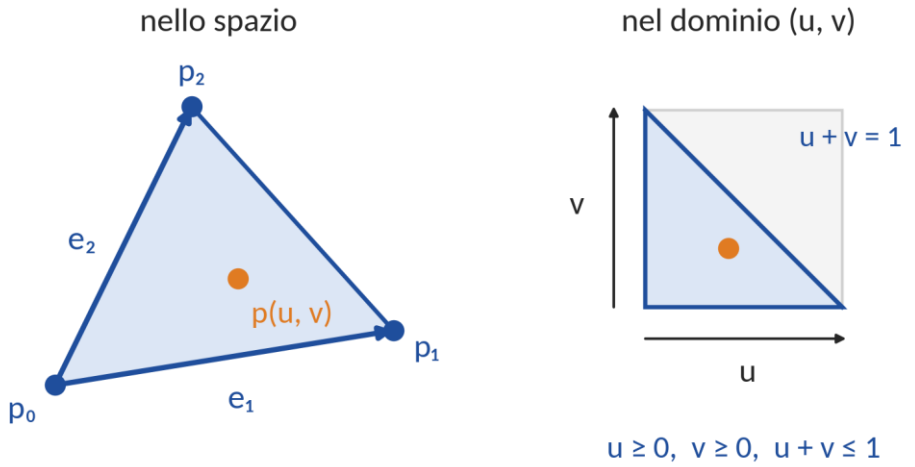
Che cosa consegnare

- `box : hittable`, con `interval` per il range di `t`
- un render della scena e uno con la camera **dentro** un box: si deve vedere la faccia interna, non lo sfondo

3 · Come si definisce un triangolo

●●● difficile

- Tre vertici, `p0`, `p1` e `p2`, e basta
- Conviene però ragionare con i due **lati** che partono da `p0`: `e1 = p1 − p0` e `e2 = p2 − p0`
- Ogni punto del triangolo è `p0 + u e1 + v e2` con u ≥ 0, v ≥ 0 e u + v ≤ 1
- I tre numeri (1 − u − v, u, v) sono le **coordinate baricentriche**: dicono quanto il punto «pesa» su ciascun vertice
- Serviranno per interpolare normali e coordinate di texture (lezioni 8 e 10)

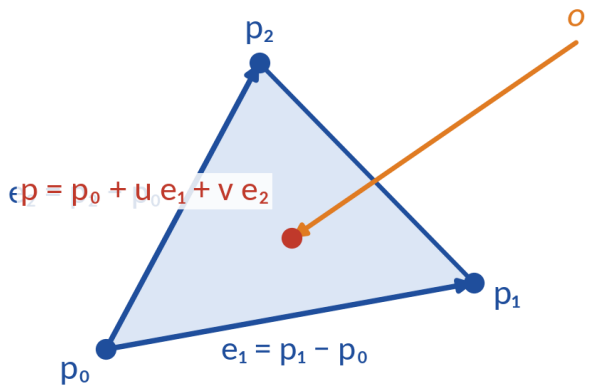


$$\mathbf{p}(u,v) = \mathbf{p}_0 + u\,\mathbf{e}_1 + v\,\mathbf{e}_2, \qquad \mathbf{e}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{e}_2 = \mathbf{p}_2 - \mathbf{p}_0, \qquad u, v \geq 0,\ u + v \leq 1$$

gli stessi (u, v) che restituirà il test di intersezione

3 · Möller–Trumbore

- Uguagliando il raggio alla parametrizzazione si ottiene un sistema di tre equazioni in (t, u, v)
- Möller–Trumbore lo risolve con la regola di Cramer, dove i tre determinanti sono prodotti misti
- Non serve la normale del piano, e non si divide finché non si sa che l'intersezione esiste; se il determinante è quasi zero il raggio è parallelo al triangolo
- Le condizioni si controllano **una alla volta**: appena u < 0 si esce, senza calcolare il resto



dentro se $u \geq 0,\ v \geq 0,\ u + v \leq 1$

$$\mathbf{e}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{e}_2 = \mathbf{p}_2 - \mathbf{p}_0, \quad \mathbf{s} = \mathbf{o} - \mathbf{p}_0$$

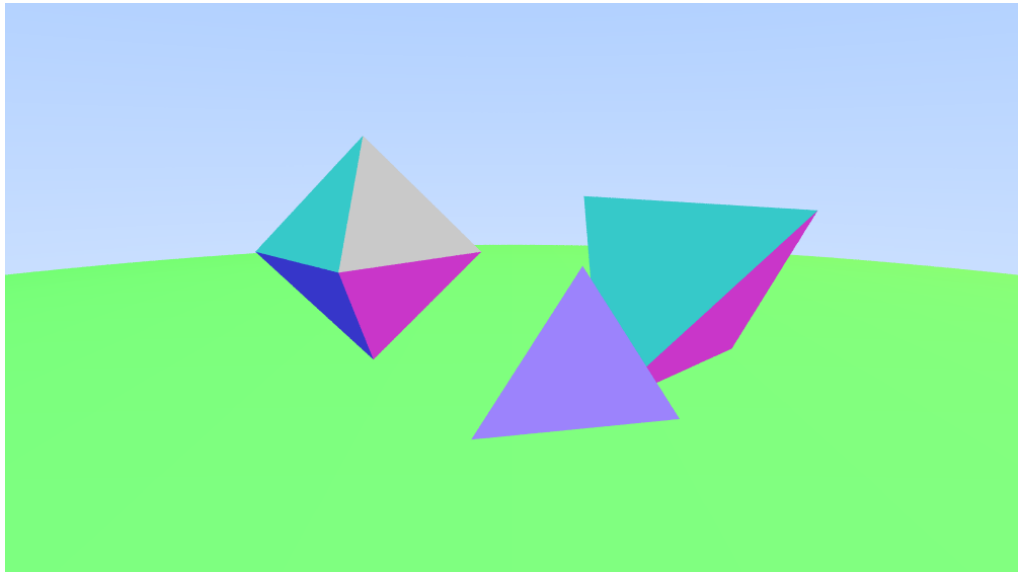
$$\det = (\mathbf{d} \times \mathbf{e}_2) \cdot \mathbf{e}_1 \qquad (\text{se } |\det| < \varepsilon \text{ il raggio è parallelo})$$

$$u = \frac{(\mathbf{d} \times \mathbf{e}_2) \cdot \mathbf{s}}{\det}, \qquad v = \frac{(\mathbf{s} \times \mathbf{e}_1) \cdot \mathbf{d}}{\det}, \qquad t = \frac{(\mathbf{s} \times \mathbf{e}_1) \cdot \mathbf{e}_2}{\det}$$

Möller, Trumbore (1997), **Fast, Minimum Storage Ray/Triangle Intersection**

3 · Due strade a confronto

- Scrivere `triangle : hittable` in **due** versioni: piano + test (α, β) come nell'homework 1, e Möller–Trumbore
- Verificare che producano la **stessa** immagine, poi misurare quale è più veloce e spiegare perché
- Costruire a mano un solido di pochi triangoli: un ottaedro sono sei vertici e otto facce
- Salvare (u, v) dentro `hit_record`: nella lezione 8 le mesh ne avranno bisogno



un ottaedro (8 triangoli) e un tetraedro (4)

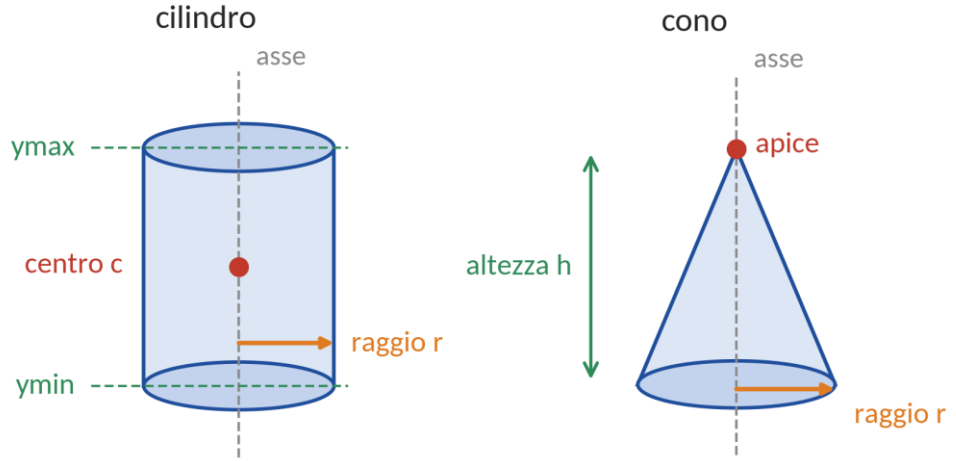
Che cosa consegnare

- le due versioni del test, con un interruttore per passare dall'una all'altra
- i tempi misurati sulla stessa scena e sullo stesso numero di raggi, e due righe di commento sul perché

4 · Come si definiscono cilindro e cono

●●● difficile

- Il **cilindro** ha quattro parametri: un asse (qui parallelo a y), un centro `c` su quell'asse, un raggio `r` e le due quote `ymin` e `ymax`
- Il **cono** ne ha quattro anche lui: un apice, un asse, il raggio di base `r` e l'altezza `h`
- Nel cilindro il raggio è costante; nel cono cresce in proporzione alla distanza dall'apice, con `k = r / h`
- Entrambe le equazioni non parlano di y: la y entra solo come **vincolo**, ed è lei a rendere la forma finita
- Così come sono, sono superfici **aperte**: per avere solidi chiusi servono i tappi



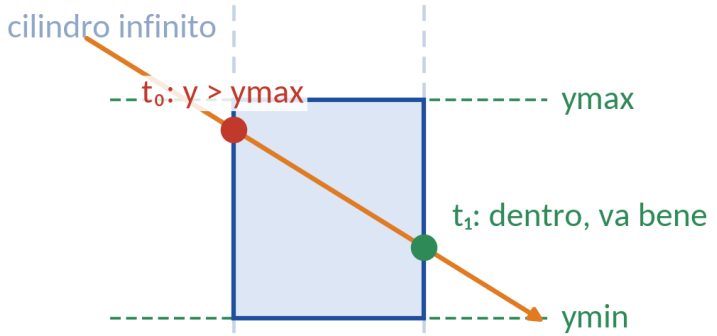
$$\text{cilindro}: \quad (x - c_x)^2 + (z - c_z)^2 = r^2 \quad \text{con} \quad y_{\min} \le y \le y_{\max}$$

$$\text{cono}: \quad x^2 + z^2 = k^2 (y - y_a)^2 \quad \text{con} \quad k = \frac{r}{h} \quad \text{(il raggio cresce con la distanza dall'apice)}$$

quattro parametri per il cilindro, quattro per il cono

4 · L'intersezione con le quadriche

- Si ottiene un'equazione di **secondo grado**, come per la sfera
- Nel cilindro la y non compare: i coefficienti usano solo le componenti x e z
- Trovate le radici, si tiene solo la parte con `ymin ≤ y ≤ ymax`: **vanno provate entrambe**
- I tappi sono due dischi: l'homework 1 li ha già risolti
- La normale è radiale sul fianco, (0, ±1, 0) sui tappi, e sul cono va ricavata dal gradiente



vanno provate entrambe le radici, poi i tappi

$$\text{cilindro:} \quad a = d_x^2 + d_z^2, \quad h = o_x d_x + o_z d_z, \quad c = o_x^2 + o_z^2 - r^2$$

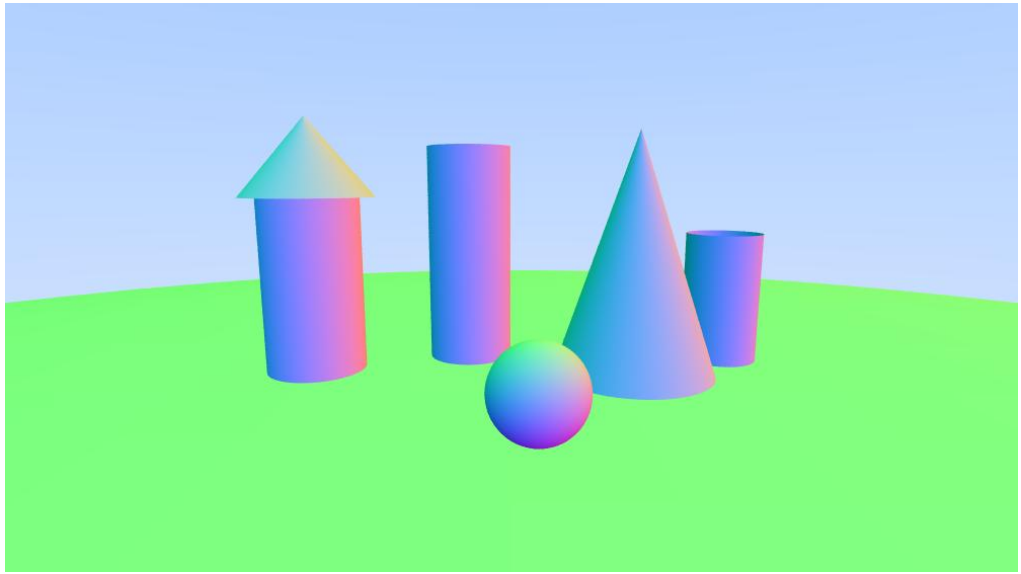
$$\text{cono:} \quad x^2 + z^2 = k^2 (y - y_a)^2, \qquad k = r/h$$

$$\text{normale sul fianco del cilindro:} \quad \mathbf{n} = \frac{(p_x - c_x,\ 0,\ p_z - c_z)}{r}$$

stessa struttura del test raggio-sfera, con in più il vincolo su y

4 · Un piccolo colonnato

- Scrivere `cylinder : hittable` (con e senza tappi) e `cone : hittable`
- Comporre una scena con tre cilindri di altezza diversa e due coni
- Provare anche il cilindro **aperto**: guardandoci dentro si deve vedere la superficie interna, non il vuoto
- **Torna utile subito**: nella II° homework servono cilindri con texture, e la mappatura cilindrica parte da qui



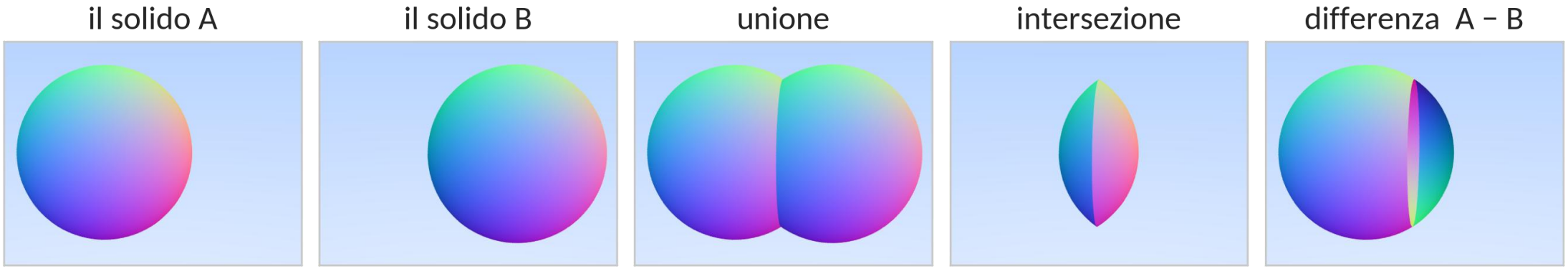
a destra un cilindro senza tappi: si vede il fianco interno

Che cosa consegnare

- `cylinder` e `cone`, con i tappi opzionali
- un render del colonnato e un primo piano del bordo di un tappo: è lì che si vedono gli errori sulla normale

5 · Unire, intersecare, sottrarre

●●● difficile



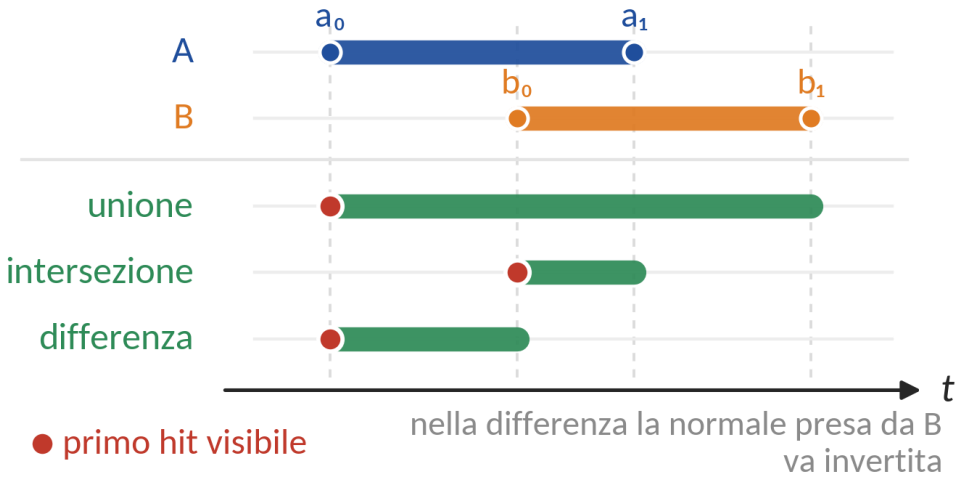
due sfere e i tre solidi che se ne ricavano, renderizzati con il solito colore dalle normali

- Un **solido** non è una superficie: è l'insieme dei punti che stanno dentro. Da due solidi A e B se ne costruiscono altri tre
- Vista **da fuori**, l'unione sembra quella che già avete: A e B nella `hittable_list` danno la stessa immagine
- Ma non è la stessa cosa: con la camera **dentro** i solidi, e quando l'unione è operando di un'altra operazione, serve quella vera
- **Intersezione** e **differenza** danno forme che con le primitive da sole non si ottengono
- La differenza non è simmetrica: `A - B` è una sfera morsicata, `B - A` è tutt'altra cosa

$$A \cup B = \{\mathbf{p} : \mathbf{p} \in A \text{ o } \mathbf{p} \in B\}, \quad A \cap B = \{\mathbf{p} : \mathbf{p} \in A \text{ e } \mathbf{p} \in B\}, \quad A \setminus B = \{\mathbf{p} : \mathbf{p} \in A \text{ e } \mathbf{p} \notin B\}$$

5 · L'algebra degli intervalli

- Finora `hit` restituisce **un punto**; per le operazioni booleane serve sapere dove il raggio **entra** ed **esce** dal solido
- Per un solido convesso è un solo intervallo [t0, t1]: di nuovo la classe `interval` della lezione 4
- Le tre operazioni sui solidi diventano tre operazioni su quegli intervalli
- Il punto visibile è il **più piccolo** t del risultato che cade dentro `ray_t`
- **Nella differenza** la superficie presa da B si vede da dentro: la normale va invertita



$$A \cup B: \ [\min(a_0, b_0),\ \max(a_1, b_1)] \quad \text{(se si toccano)}$$
$$A \cap B: \ [\max(a_0, b_0),\ \min(a_1, b_1)] \quad \text{vuoto se } \ b_0 > a_1$$
$$A \setminus B: \ [a_0, a_1] \text{ tolto } [b_0, b_1] \longrightarrow \text{fino a due intervalli}$$

l'intersezione di convessi resta un intervallo; la differenza può spezzarsi in due

5 · L'unione: fondere gli intervalli

- Un solido non dà più «il primo punto», ma la **lista degli intervalli** in cui il raggio gli sta dentro
- Per l'unione: si mettono in fila gli intervalli di A e di B, si **ordinano** per t di ingresso, si fondono quelli che si toccano
- L'ingresso è quello che arriva **prima**, l'uscita quella che arriva **dopo**, ciascuno con la sua normale
- Se A e B non si toccano restano **due** intervalli: per questo serve una lista

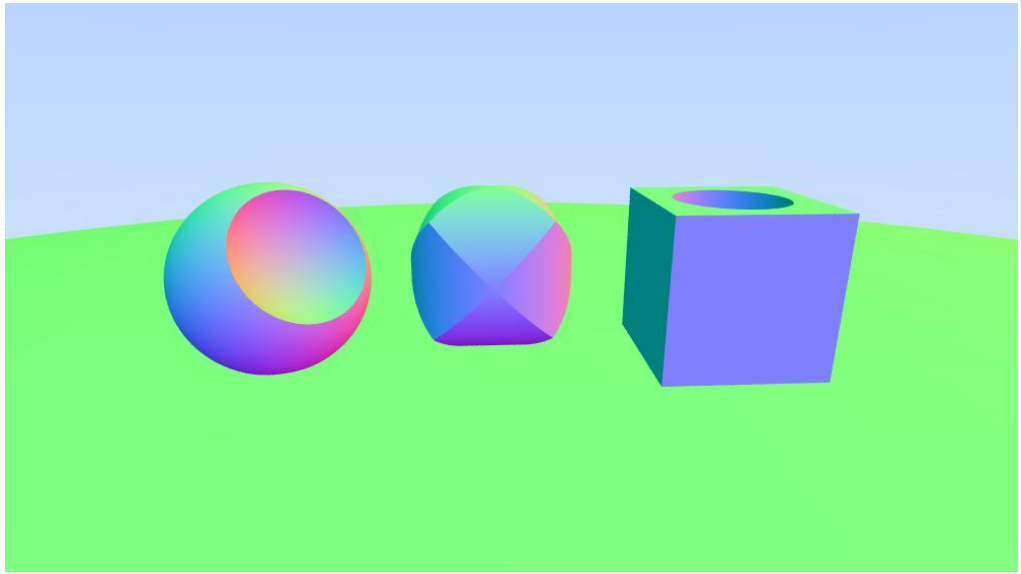



```
out.push_back(all[0]);                  // all[] ordinato per t
for (size_t i = 1; i < all.size(); ++i) {
  span& last = out.back();
  if (all[i].in.t > last.out.t)         // staccati: restano due
    out.push_back(all[i]);
  else if (all[i].out.t > last.out.t) // si toccano: fondo
    last.out = all[i].out;              // vince la piu' lontana
}
```

`hit` si scrive una volta sola: scorre gli intervalli e prende il primo t dentro `ray_t`

5 · Tre forme da costruire

- Aggiungere a `hittable` un metodo che restituisce l'intervallo attraversato, e implementarlo per sfera, box e cilindro
- Scrivere `csg : hittable` con le tre operazioni
- Costruire una sfera **morsicata**, un **dado forato** (box meno cilindro) e il **solido di Steinmetz** (tre cilindri ortogonali intersecati)
- Il caso che fa sbagliare è quello in cui il foro taglia il solido in **due** pezzi lungo il raggio



sfera − sfera, tre cilindri intersecati, box − cilindro

Che cosa consegnare

- `csg : hittable` con unione, intersezione e differenza
- i tre solidi in un unico render, più un'immagine con la camera che entra dentro la parte scavata

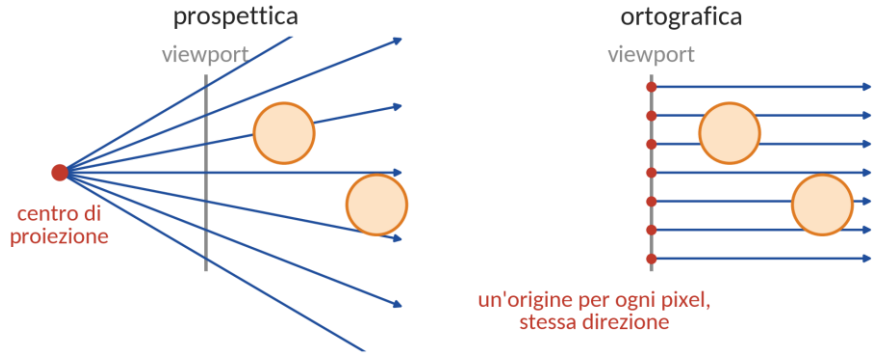
Homeworks

- Nuove primitive
- **La camera**
- Qualità e prestazioni
- Modalità di consegna

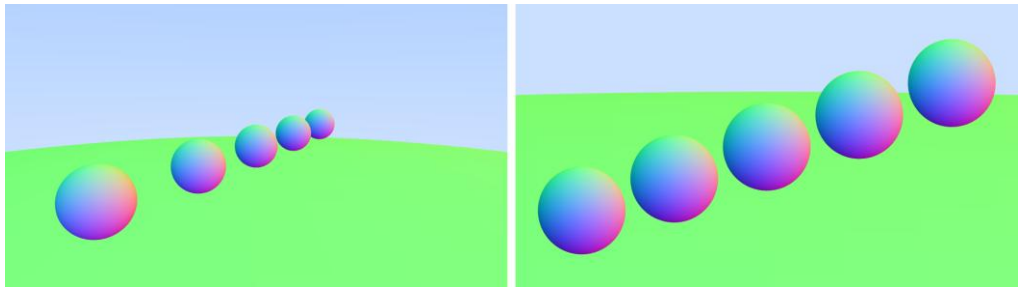
6 · La camera ortografica

●○○ facile

- Nella proiezione parallela tutti i raggi hanno la **stessa direzione**: cambia l'origine, non la direzione
- Sparisce il field of view: l'inquadratura si dà in unità del mondo (`ortho_height`)
- In `get_ray` l'origine diventa il punto campionato sul viewport e la direzione è `−w`
- Inquadrare la stessa scena con le due camere e spiegare perché con quella parallela le sfere lontane **non rimpiccioliscono**



$$\mathbf{o} = \mathbf{s}_{ij}, \qquad \mathbf{d} = -\mathbf{w} \qquad \text{contro} \qquad \mathbf{o} = \mathbf{c}, \qquad \mathbf{d} = \mathbf{s}_{ij} - \mathbf{c}$$



a sinistra prospettica, a destra ortografica

Che cosa consegnare

- la camera con un interruttore fra le due proiezioni, senza duplicare il codice di `render`
- le due immagini della **stessa** scena e tre righe sul perché la seconda sembra «piatta»

7 · Un panorama a 360°

●●○ medio

- Qui il viewport sparisce del tutto: la direzione del raggio si costruisce da **due angoli**
- La colonna del pixel dà l'azimut ϕ (da 0 a 2π), la riga dà l'inclinazione θ (da 0 a π)
- L'immagine deve essere **2:1**, altrimenti la proiezione non torna
- Ai poli la distorsione è enorme: un oggetto allo zenit si spalma su tutta la riga in alto

$$\theta = \pi \frac{j + 0.5}{\texttt{image_height}}, \qquad \varphi = 2\pi \frac{i + 0.5}{\texttt{image_width}}$$

$$\mathbf{d} = \sin\theta \sin\varphi\ \mathbf{u} \ + \ \cos\theta\ \mathbf{v} \ - \ \sin\theta \cos\varphi\ \mathbf{w}$$

una direzione per ogni pixel

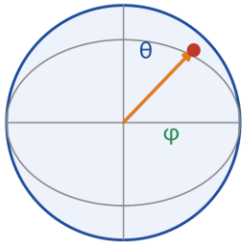
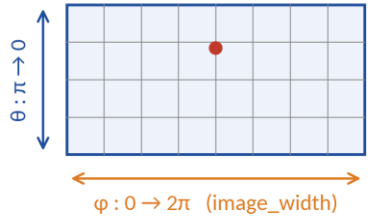
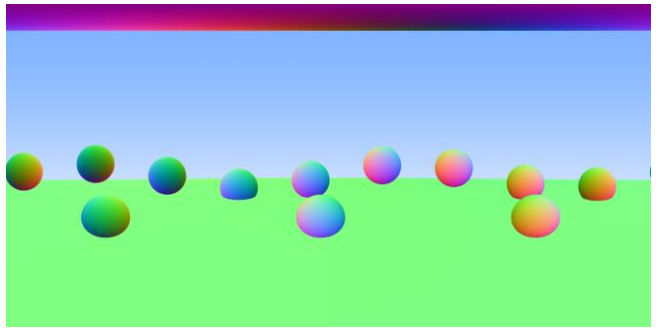


immagine 2:1 (equirettangolare)





si apre in un visore 360°, anche quello del telefono

Che cosa consegnare

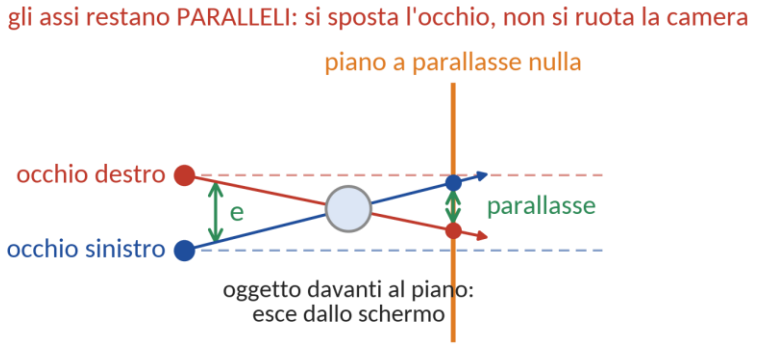
- la camera equirettangolare come terza modalità, accanto a prospettica e ortografica
- il panorama 2:1 di una scena con oggetti **tutto intorno** alla camera, e una frase su dove si vede la distorsione

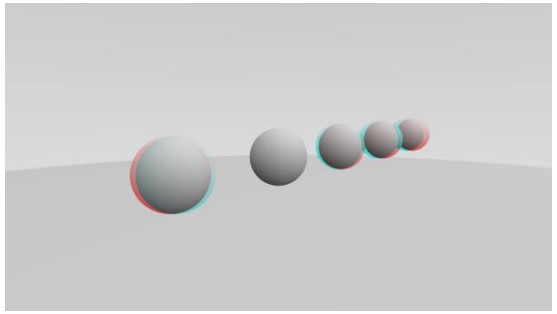
8 · La scena in 3D stereoscopico

●●○ medio

- Due immagini della stessa scena, una per occhio, separate dalla distanza interoculare **e**
- **Gli assi restano paralleli**: si trasla l'occhio e si lascia il viewport dov'è. Ruotare le due camere l'una verso l'altra introduce parallasse verticale e fa venire il mal di testa
- Gli oggetti alla distanza di convergenza hanno parallasse nulla: stanno «sullo schermo»
- L'anaglifo si compone prendendo il rosso dall'occhio sinistro, verde e blu dal destro

$$\mathbf{c}_L = \mathbf{c} - \tfrac{e}{2}\,\mathbf{u}, \qquad \mathbf{c}_R = \mathbf{c} + \tfrac{e}{2}\,\mathbf{u}, \qquad \text{il viewport resta dov'è}$$





con gli occhiali rosso/ciano le sfere vanno in profondità

Che cosa consegnare

- l'anaglifo di una scena con oggetti a distanze **molto** diverse, e tre valori di e a confronto
- conviene convertire in grigio prima di comporre i canali: con le immagini a colori il rosso saturo «buca» l'occhio sbagliato

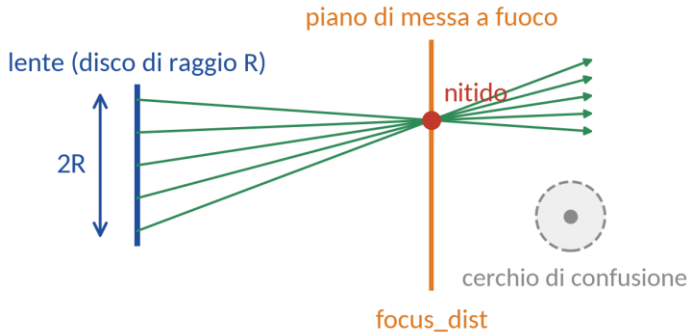
9 · Profondità di campo

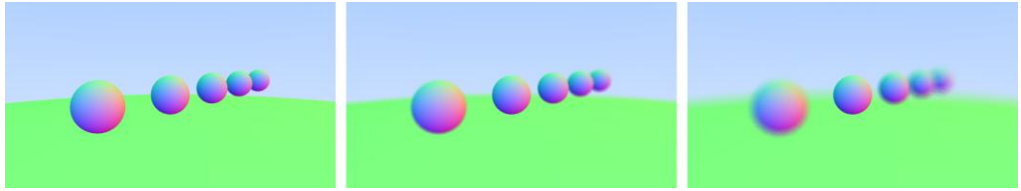
●●○ medio

- Finora la camera è un foro senza dimensioni e tutto è a fuoco. Una lente vera ha un **diametro**
- L'origine del raggio non è più un punto: si campiona un punto a caso su un disco
- La direzione punta sempre allo stesso punto del **piano di messa a fuoco**, che quindi resta nitido
- Serve molto antialiasing: con pochi campioni lo sfocato diventa rumore. Nelle immagini qui a fianco sono 96

$$(r_x, r_y) \in \text{disco di raggio } R = \texttt{focus_dist} \cdot \tan \frac{\texttt{defocus_angle}}{2}$$

$$\mathbf{o} = \mathbf{c} + r_x \mathbf{u} + r_y \mathbf{v}, \qquad \mathbf{d} = \mathbf{s}_{ij} - \mathbf{o}$$





`defocus_angle` = 0°, 2,5°, 7°: a fuoco sempre la seconda sfera

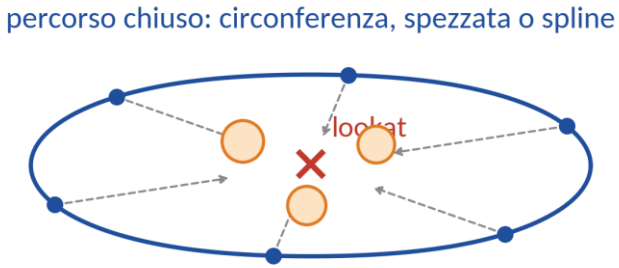
Che cosa consegnare

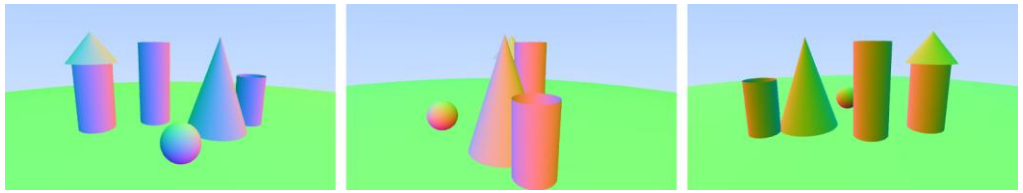
- `defocus_angle` e `focus_dist` nella classe `camera`, con `get_ray` modificato
- tre render ad apertura crescente in cui **lo stesso** oggetto resta nitido: è la prova che il piano di fuoco è quello giusto

10 · Animazione della camera

●●● difficile

- Definire un **percorso chiuso**: una circonferenza, una spezzata o una spline di Catmull–Rom
- A ogni passo cambiano `lookfrom` e, se volete, anche `lookat`: il resto si ricalcola in `initialize()`
- La parte interessante è la **velocità costante**: va parametrizzato per lunghezza d'arco, altrimenti la camera accelera nelle curve
- **Occhio al caso degenere**: se il percorso passa sopra il soggetto, `vup` diventa parallelo a `w` e l'immagine esplode (lezione 5)





tre istanti dello stesso percorso circolare

Che cosa consegnare

- il codice che genera i frame numerati e il video montato, almeno 10 secondi a 25 frame al secondo
- se il rendering in tempo reale non regge, salvate i frame e montateli dopo: va benissimo lo stesso

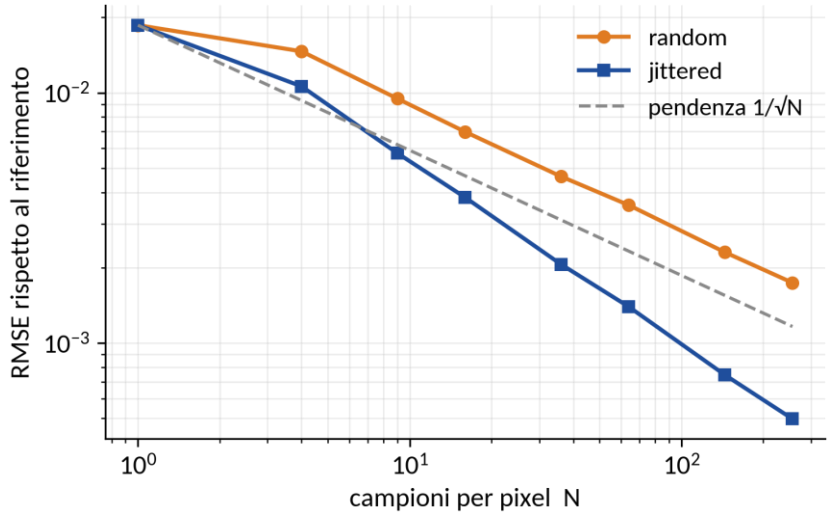
Homeworks

- Nuove primitive
- La camera
- **Qualità e prestazioni**
- Modalità di consegna

11 · Antialiasing: quanto serve davvero?

●●○ medio

- Implementare i tre schemi della lezione 5: uniforme, random e jittered su griglia √N × √N
- Prendere come **riferimento** un render a 1024 campioni e misurare l'errore quadratico medio degli altri
- Riportare l'errore in scala doppio logaritmica al crescere di N e misurare la pendenza
- **Il risultato atteso**: random scende come 1/√N, cioè con pendenza −0,5; jittered scende **più in fretta**



$$\mathrm{RMSE}(N) = \sqrt{\frac{1}{3WH}\sum_{\text{pixel}} \|\mathbf{c}_N - \mathbf{c}_{\text{rif}}\|^2} \quad \Longrightarrow \quad \mathrm{RMSE} \propto N^{-1/2}$$



N = 1, 4, 64 sullo stesso bordo

Che cosa consegnare

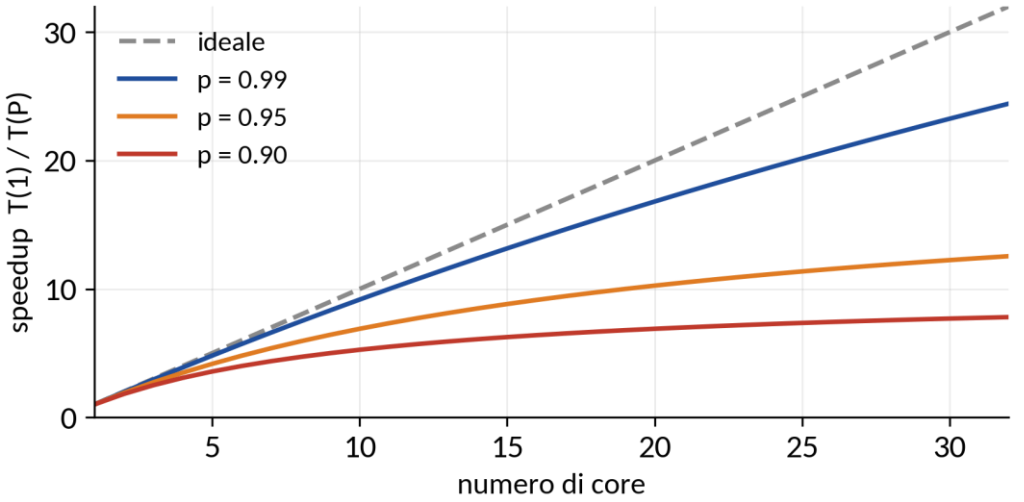
- il grafico dell'errore con le curve dei tre schemi e la pendenza misurata
- una riga di conclusione: **a parità di errore**, quanti campioni random servono per arrivare dove arriva jittered?

12 · Quanto è parallelo il renderer?

●●○ medio

- Misurare il tempo di `render` e di `parallel_render` al variare del numero di thread disponibili
- Tracciare **speedup** ed **efficienza** e confrontarli con `std::thread::hardware_concurrency()`
- Stimare dalla curva la frazione parallelizzabile p con la legge di Amdahl: di solito si scopre che non è 1
- **Usate** `std::chrono::steady_clock`: `time` e `difftime` hanno risoluzione di **un secondo**
- Provare due suddivisioni: una riga per task e blocchi di righe

$$S(P) = \frac{T(1)}{T(P)} = \frac{1}{(1-p)+p/P}, \qquad E(P) = \frac{S(P)}{P}$$



con p = 0,95 oltre i venti core non si guadagna quasi più niente

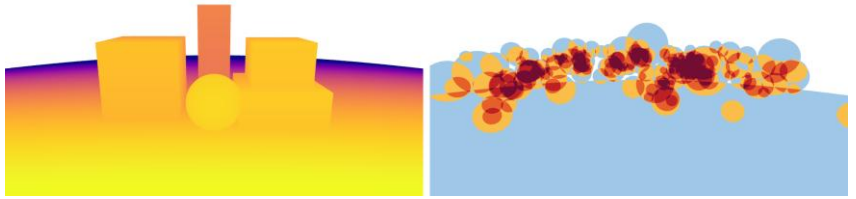
Che cosa consegnare

- la tabella dei tempi e il grafico dello speedup della **vostra** macchina, con il numero di core dichiarato
- il valore di p stimato e un commento: dove se ne va la parte seriale?

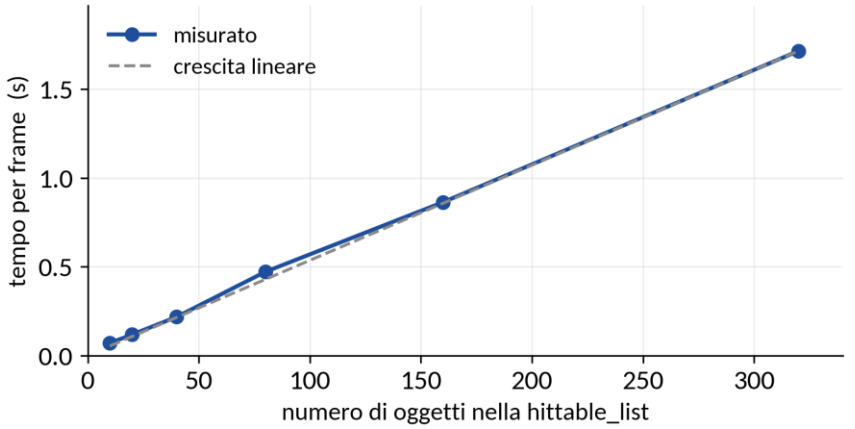
13 · Viste di debug

●○○ facile

- Aggiungere al renderer tre modalità selezionabili da tastiera, oltre a quella normale
- **Profondità**: il pixel colorato in base a `t` con una scala di colori. Attenzione: `t` non è una distanza se `d` non è normalizzato
- **Normali**: `0.5 * (n + 1)`, che già avete; serve da controllo
- **Costo**: quanti oggetti il raggio colpisce davvero: le zone calde sono dove il renderer fatica
- **La scoperta**: il numero di **test** è invece sempre lo stesso, pari al numero di oggetti. È il problema che risolve il BVH della lezione 9



profondità e costo per pixel

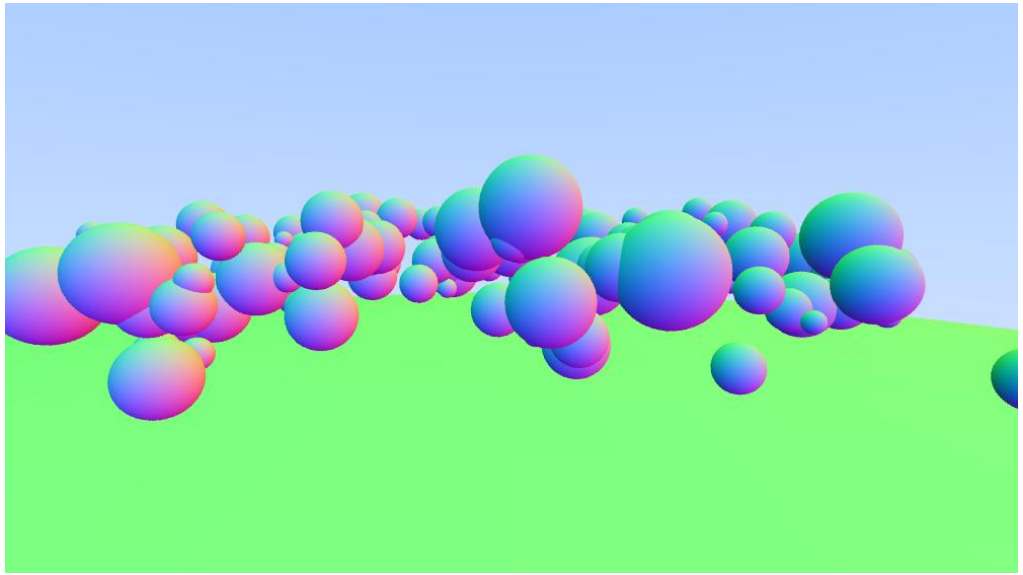


Che cosa consegnare

- le tre viste della stessa scena e il grafico del tempo al crescere del numero di oggetti (10, 50, 100, 500 sfere)
- una riga: la curva è una retta? E che cosa vi aspettate che succeda con un BVH?

Fuori concorso: la scena più bella

- Con quello che avete — sfere, le forme nuove, il colore dalle normali e una camera che si può mettere ovunque — si può già fare molto
- Chi vuole consegna **una sola immagine** in più, senza vincoli di scena, di risoluzione o di buon gusto
- Non deve essere fisicamente sensata: deve essere bella
- Le tre più votate in aula apriranno la lezione 6



140 sfere, nessun materiale: solo normali e uno sfondo a gradiente

Una regola sola

- l'immagine deve uscire dal **vostro** renderer, non da un'altra applicazione

Consigli pratici

- Partite dal progetto della lezione 5 e tenetene una copia funzionante prima di toccare il codice
- Una classe per file header, come nel resto del corso: il renderer non deve cambiare, cambiano solo gli oggetti che gli date
- Controllate sempre i tre casi limite: raggio **parallelo**, `t` **negativo**, raggio che parte **dentro** l'oggetto
- Usate `interval` per il range di `t` e `set_face_normal` per la faccia: sono lì per questo
- Se un'immagine è nera, guardate il verso della normale; se è «bucata», il range di `t`
- Lavorate a bassa risoluzione finché il test non è corretto: un render a 200×180 con un campione esce in un attimo

Se vi bloccate

- scrivete pure, ma prima provate a stampare `t`, la normale e il punto di intersezione per **un solo** pixel: nove volte su dieci l'errore salta fuori da solo

Homeworks

- Nuove primitive
- La camera
- Qualità e prestazioni
- **Modalità di consegna**

Modalità di consegna

- Consegna **solo** attraverso la piattaforma Moodle
- Scadenza: **da definire**
- Per ogni homework consegnare:
  - i sorgenti, cioè il progetto Visual Studio completo, **senza** le cartelle `x64`, `Debug`, `Release` e `packages`
  - l'eseguibile
  - 2–3 render di esempio e, dove richiesto, il video
  - un `README.txt` di una pagina: che cosa avete implementato, che cosa non funziona, dove vi siete bloccati
- Un archivio per homework, con nome `Cognome_Nome_HW1_n.zip`
- Si discute tra di voi, ma si lavora e si consegna **singolarmente**

Come viene valutato

- che il test di intersezione sia **corretto**, compresi i casi limite — non che la scena sia complicata
- che il codice si integri nel renderer esistente invece di duplicarlo
- che il `README` dica la verità su quello che non funziona